

Создание Dockerfile and Compose

Задание 1

1. Создаю пустую директорию docker-hw24 по пути /home/puppy. В которой создаю конфиг файл nginx со своими параметрами:

```
user www-data;
worker_processes auto;
worker_cpu_affinity auto;
pid /run/nginx.pid;
error_log /var/log/nginx/error.log;
include /etc/nginx/modules-enabled/*.conf;

events {
    worker_connections 1024;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;

    server {
        listen 8080;
        server_name localhost;

        root /app/static;
        index index.nginx-site.html index.html;

        location / {
            try_files $uri $uri/ =404;
        }
    }
}
```

2. Также еще внутри директории docker-hw24 создаю директорию static, откуда файл конфигурации будет брать статические файлы по типу index.html и тд. Как выглядит структура:

```
puppy@dev:~/docker-hw24$ tree
.
├── Dockerfile
├── nginx.conf
└── static
    └── index.nginx-site.html
```

3. Затем в папке static создаю файл index.nginx-site.html с настройкой собственного сайта:

```
puppy@dev:~/docker-hw24$ cat static/index.nginx-site.html
<!DOCTYPE html>
<html>
<head>
<title>Docker</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Привет! Ты внутри докера</h1>
<p>Веб сервер работает на порту 8080</p>
</body>
</html>
```

4. Далее в этой директории создаю файл Dockerfile с следующим содержимым:

```
puppy@dev:~/docker-hw24$ cat Dockerfile
FROM ubuntu:20.04

RUN apt-get update && \
apt-get install -y \
curl \
nginx

RUN mkdir -p /app/static/

WORKDIR /app/static/

COPY nginx.conf /etc/nginx/nginx.conf

COPY static/ /app/static/

EXPOSE 8080

CMD ["nginx", "-c", "/etc/nginx/nginx.conf", "-g", "daemon off;"]
```

5. Сохраняю файл и запускаю сборку с помощью команды `docker build -t my-nginx-app /home/puppy/docker-hw24/`. Решила указать имя, так как при обычном запуске, отрабатывает нормально, но не отображается в списках образов:

```
puppy@dev:~$ docker build -t my-nginx-app docker-hw24/
[+] Building 4.0s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 311B                                0.0s
=> [internal] load metadata for docker.io/library/ubuntu:20.04    2.3s
=> [internal] load .dockerignore                                   0.0s
=> => transferring context: 2B                                       0.0s
=> [1/6] FROM docker.io/library/ubuntu:20.04@sha256:8feb4d8ca5354def3d8fce243717141ce31e2c428701f6682bd2fafa15388214 0.5s
=> => resolve docker.io/library/ubuntu:20.04@sha256:8feb4d8ca5354def3d8fce243717141ce31e2c428701f6682bd2fafa15388214 0.5s
=> [internal] load build context                                   0.2s
=> => transferring context: 106B                                       0.0s
=> CACHED [2/6] RUN apt-get update && apt-get install -y curl nginx 0.0s
=> CACHED [3/6] RUN mkdir -p /app/static/                          0.0s
=> CACHED [4/6] WORKDIR /app/static/                                0.0s
=> CACHED [5/6] COPY nginx.conf /etc/nginx/nginx.conf              0.0s
=> CACHED [6/6] COPY static/ /app/static/                          0.0s
=> exporting to image                                              0.8s
=> => exporting layers                                                0.0s
=> => exporting manifest sha256:8977c5e308a74d17b08afe89d176ff89c2f1f571204688f1989c7de5dbde8ed1 0.0s
=> => exporting config sha256:438ace790d4f814d7df7a6b931ee51f1fe30b65835b8e34f193907838a40dab0 0.0s
=> => exporting attestation manifest sha256:92f4a175a7653cfe4c6262c67b45a2eae2423ea1a876792b64ef4b78ea2071f2 0.0s
=> => exporting manifest list sha256:f9af8e4d848b2fa61d6dbc5c2fea3a03121cbdba8bfa2fa4e9353cf4cbb2ebc5 0.0s
=> => naming to docker.io/library/my-nginx-app:latest              0.0s
=> => unpacking to docker.io/library/my-nginx-app:latest           0.1s

puppy@dev:~$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
my-nginx-app:latest	f9af8e4d848b	315MB	96.1MB	

6. Теперь запускаю контейнер из созданного образа `docker run -d --name test-nginx -p 8080:8080 my-nginx-app:latest`. После чего проверяю с помощью `curl http://localhost:8080:`

```
puppy@dev:~$ docker run -d --name test-nginx -p 8080:8080 my-nginx-app:latest
742d86f927cd251a973c5980c84b281c3de59df70613aba1e3e46ac6c95761e6
puppy@dev:~$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Docker</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Привет! Ты внутри докера</h1>
<p>Веб сервер работает на порту 8080</p>
</body>
</html>
```

Задание 2

1. Создаю пустую директорию docker-compose
2. Также создаю файл среды .env, нужен, чтобы данные авторизации как в моем случае были не в открытом виде, а подтягивались отдельно:

```
puppy@dev:~/docker-compose$ cat .env
POSTGRES_DB=app
POSTGRES_USER=puppy
POSTGRES_PASSWORD=1
```

3. Теперь внутри директории создаю файл docker-compose.yml со следующим содержимым:

```
services:
  db:
    image: postgres:latest
    container_name: postgresql
    restart: unless-stopped

    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}

    networks:
      - nginx_network

    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5

  web:
    image: nginx:latest
    container_name: nginx_app
    restart: unless-stopped

    depends_on:
      db:
        condition: service_healthy

    links:
      - db

    ports:
      - "80:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
      - ./static:/var/www/html/

    networks:
      - nginx_network

volumes:
  nginx_data:

networks:
  nginx_network:
    driver: bridge
```

4. Запускаю проект с помощью команды docker compose up -d --build:

```
puppy@dev:~/docker-compose$ docker compose up -d --build
[+] up 3/3
✓ Network docker-compose_nginx_network Created
✓ Container postgresql Healthy
✓ Container nginx_app Started
```

```
puppy@dev:~/docker-compose$ docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
nginx_app	nginx:latest	"/docker-entrypoint..."	web	27 seconds ago	Up 15 seconds	0.0.0.0:80->80/tcp, [::]:80->80/tcp
postgresql	postgres:latest	"docker-entrypoint.s..."	db	27 seconds ago	Up 26 seconds (healthy)	5432/tcp

5. Проверяю работоспособность приложения с помощью команды `curl http://localhost:80:`

```
puppy@dev:~/docker-compose$ curl http://localhost:80
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Docker</title>
</head>
<body>
<h1>Привет! Ты внутри докера</h1>
<p>Веб сервер работает на порту 80</p>
</body>
</html>
```

6. Если в веб-версии:

